

OS作业8-郭威威-2414308

基于多线程与Posix信号量的睡眠理发师问题实现

实验报告

班级： 24二学士 姓名： 郭威威 学号： 2414308 日期： 2025/11/28

一、实验基本信息

项目	内容
实验名称	基于多线程与Posix信号量的睡眠理发师问题实现
实验目的	1. 掌握Posix信号量与Pthread互斥量的使用方法； 2. 理解多线程同步与互斥机制，解决临界资源竞争； 3. 实现睡眠理发师经典同步问题，满足场景约束与输出要求； 4. 避免竞态条件与忙等待，保障程序正确性与高效性。
实验环境	操作系统：Linux（Ubuntu 20.04 LTS） 编译器：GCC 9.4.0 IDE：Code::Blocks 20.03（可选） 同步机制：Posix信号量、Pthread互斥量 依赖库：Pthread线程库
实验工具	GCC编译器、Code::Blocks IDE、Linux终端

二、实验原理

（一）睡眠理发师问题核心描述

理发店包含1名理发师、1把理发椅和3把等待椅。核心逻辑如下：

- 1. 无顾客时，理发师在理发椅上睡眠；
- 2. 顾客到达后，若理发师空闲则唤醒理发师理发；
- 3. 若理发师忙碌，顾客可占用等待椅（最多3人），无等待椅则直接离开；
- 4. 累计30名顾客，全部服务完成后理发师停止工作。

该问题的核心矛盾是**线程同步**（理发师与顾客的唤醒/等待）和**资源互斥**（等待椅的竞争），需通过信号量与互斥量解决。

（二）关键技术原理

- 5. **Posix信号量**：用于线程间同步，通过P/V操作控制信号量值，实现线程阻塞与唤醒，避免忙等待。
 - `sem_init(sem_t *sem, int shared, unsigned int value)`：初始化信号量，`shared=0` 表示线程间共享，`value` 为初始值；
 - `sem_wait(sem_t *sem)`：P操作，信号量值>0则减1，否则阻塞；
 - `sem_post(sem_t *sem)`：V操作，信号量值加1，唤醒阻塞线程；
 - `sem_destroy(sem_t *sem)`：销毁信号量，释放资源。
- 6. **Pthread互斥量**：用于保护临界资源（如等待顾客数），确保同一时间仅一个线程访问临界区，避免竞态条件。
 - `pthread_mutex_init(pthread_mutex_t *mutex, NULL)`：初始化互斥量；
 - `pthread_mutex_lock(pthread_mutex_t *mutex)`：加锁，未获取锁则阻塞；
 - `pthread_mutex_unlock(pthread_mutex_t *mutex)`：解锁，释放临界区；
 - `pthread_mutex_destroy(pthread_mutex_t *mutex)`：销毁互斥量。
- 7. **随机函数**：`srand((unsigned int)time(NULL))` 初始化随机数种子，`rand()` 生成随机整数，用于模拟顾客到达间隔（1-3秒）和理发时长（1-10秒，平均5秒）。
- 8. **多线程编程**：`pthread_create()` 创建线程，`pthread_join()` 等待线程退出，`pthread_exit()` 线程主动退出，实现理发师与顾客的并发执行。

（三）同步机制设计

同步组件	作用	初始值

信号量 <code>customer</code>	顾客到达信号，用于唤醒睡眠的理发师	0
信号量 <code>barber</code>	理发师可用信号，标识理发师是否空闲（1=空闲，0=忙碌）	1
互斥量 <code>mutex</code>	保护临界资源 <code>waiting</code> （等待顾客数），避免多线程同时修改	-
全局变量 <code>waiting</code>	记录当前等待理发的顾客数，范围0-3（等待椅数量为3）	0

三、实验步骤

（一）环境搭建

- 9. 安装编译工具：打开Linux终端，执行命令 `sudo apt-get install build-essential`，安装GCC编译器及依赖库；
- 10. 安装IDE（可选）：执行命令 `sudo apt-get install codeblocks`，安装Code::Blocks IDE；
- 11. 验证环境：输入 `gcc --version`，若输出GCC版本信息（如 `gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0`），则环境搭建成功。

（二）代码编写

- 12. 新建C文件 `sleeping_barber.c`，引入头文件：

代码块

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <time.h>
6  #include <unistd.h>
```

- 13. 定义全局常量与同步组件：

代码块

```
1  #define CHAIRS 3           // 等待椅数量
```

```

2  #define TOTAL_CUSTOMERS 30 // 总顾客数
3  sem_t customer;           // 顾客到达信号量（唤醒理发师）
4  sem_t barber;             // 理发师可用信号量
5  pthread_mutex_t mutex;    // 保护等待顾客数的互斥量
6  int waiting = 0;          // 当前等待顾客数
7  int barber_id = 1;        // 理发师编号

```

14. 实现理发师线程函数：

代码块

```

1  void *barber_thread(void *arg) {
2      while (1) {
3          // 等待顾客到达（无顾客则阻塞睡眠）
4          printf("理发师%d线程准备进入临界区（等待顾客）\n", barber_id);
5          sem_wait(&customer);
6          printf("理发师%d线程已进入临界区（等待顾客）\n", barber_id);
7
8          // 临界区：修改等待顾客数
9          pthread_mutex_lock(&mutex);
10         waiting--;
11         printf("理发师%d被唤醒，当前等待顾客数：%d，等待椅空闲数：%d\n",
12             barber_id, waiting, CHAIRS - waiting);
13         pthread_mutex_unlock(&mutex);
14
15         // 模拟理发过程（随机时长1-10秒）
16         int cut_time = rand() % 10 + 1;
17         printf("理发师%d开始为顾客理发，预计时长：%d秒\n", barber_id, cut_time);
18         sleep(cut_time);
19         printf("理发师%d完成理发\n", barber_id);
20
21         // 释放理发师可用信号
22         sem_post(&barber);
23         printf("理发师%d线程已离开临界区（完成理发）\n", barber_id);
24
25         // 服务完30名顾客后退出
26         static int served_count = 0;
27         served_count++;
28         if (served_count == TOTAL_CUSTOMERS) {
29             printf("理发师%d已服务所有%d名顾客，下班！\n", barber_id,
TOTAL_CUSTOMERS);
30             break;
31         }
32     }
33     pthread_exit(NULL);
34 }

```

15. 实现顾客线程函数：

代码块

```
1  void *customer_thread(void *arg) {
2      int id = *(int *)arg;
3      free(arg); // 释放动态分配的编号内存
4
5      // 临界区：进店判断等待椅是否空闲
6      printf("顾客%d线程准备进入临界区（进店）\n", id);
7      pthread_mutex_lock(&mutex);
8      printf("顾客%d线程已进入临界区（进店）\n", id);
9
10     if (waiting < CHAIRS) {
11         // 有等待椅，加入等待队列
12         waiting++;
13         printf("顾客%d进店，当前等待顾客数：%d，等待椅空闲数：%d，进入等待队列\n",
14             id, waiting, CHAIRS - waiting);
15         pthread_mutex_unlock(&mutex);
16
17         // 唤醒理发师（若睡眠）
18         sem_post(&customer);
19         // 等待理发师空闲
20         sem_wait(&barber);
21
22         // 开始接受理发服务
23         printf("顾客%d开始接受理发师%d服务\n", id, barber_id);
24     } else {
25         // 无等待椅，直接离店
26         printf("顾客%d进店，无空闲等待椅，立即离店\n", id);
27         pthread_mutex_unlock(&mutex);
28         printf("顾客%d线程已离开临界区（离店）\n", id);
29         pthread_exit(NULL);
30     }
31
32     // 理发完成，离店
33     printf("顾客%d理发完成，离店\n", id);
34     printf("顾客%d线程已离开临界区（离店）\n", id);
35     pthread_exit(NULL);
36 }
```

16. 实现主函数（初始化、创建线程、资源销毁）：

代码块

```
1  int main() {
```

```

2     pthread_t barber_tid;
3     pthread_t customer_tids[TOTAL_CUSTOMERS];
4     int i;
5
6     // 初始化信号量与互斥量
7     if (sem_init(&customer, 0, 0) == -1 || sem_init(&barber, 0, 1) == -1) {
8         perror("sem_init failed");
9         exit(EXIT_FAILURE);
10    }
11    if (pthread_mutex_init(&mutex, NULL) == -1) {
12        perror("pthread_mutex_init failed");
13        exit(EXIT_FAILURE);
14    }
15
16    // 初始化随机数种子 (确保每次运行随机序列不同)
17    srand((unsigned int)time(NULL));
18
19    // 创建理发师线程
20    if (pthread_create(&barber_tid, NULL, barber_thread, NULL) == -1) {
21        perror("pthread_create barber failed");
22        exit(EXIT_FAILURE);
23    }
24    printf("理发师%d线程创建成功\n", barber_id);
25
26    // 创建30个顾客线程 (随机间隔1-3秒到达)
27    for (i = 0; i < TOTAL_CUSTOMERS; i++) {
28        int *id = malloc(sizeof(int));
29        *id = i + 1; // 顾客编号1-30
30        if (pthread_create(&customer_tids[i], NULL, customer_thread, id) ==
-1) {
31            perror("pthread_create customer failed");
32            exit(EXIT_FAILURE);
33        }
34        printf("顾客%d线程创建成功\n", *id);
35        sleep(rand() % 3 + 1); // 顾客随机到达间隔
36    }
37
38    // 等待所有顾客线程结束
39    for (i = 0; i < TOTAL_CUSTOMERS; i++) {
40        if (pthread_join(customer_tids[i], NULL) == -1) {
41            perror("pthread_join customer failed");
42            exit(EXIT_FAILURE);
43        }
44    }
45
46    // 等待理发师线程结束
47    if (pthread_join(barber_tid, NULL) == -1) {

```

```
48         perror("pthread_join barber failed");
49         exit(EXIT_FAILURE);
50     }
51
52     // 销毁信号量与互斥量，释放资源
53     sem_destroy(&customer);
54     sem_destroy(&barber);
55     pthread_mutex_destroy(&mutex);
56
57     printf("所有线程执行完毕，程序正常退出\n");
58     return 0;
59 }
```

（三）编译与运行

17. 编译代码：终端进入代码所在目录，执行命令：

代码块

```
1 gcc sleeping_barber.c -o sleeping_barber -lpthread
```

18. 运行程序：执行命令 `./sleeping_barber`，观察终端输出；

19. 验证结果：检查输出是否符合以下要求：

- 临界区进出日志完整（准备进入、已进入、已离开）；
- 理发师/顾客关键状态日志清晰（进店、等待、理发开始/结束、离店）；
- 等待椅状态实时更新（等待顾客数、空闲数）；
- 无竞态条件、无忙等待；
- 30名顾客全部处理后程序正常退出。

四、实验结果与分析

（一）运行结果示例截图（关键片段）

```
gww@gww-VMware-Virtual-Platform:~/os/第八次$ vim sleeping_barber.c
gww@gww-VMware-Virtual-Platform:~/os/第八次$ gcc sleeping_barber.c -o sleeping_barber -lpthread
gww@gww-VMware-Virtual-Platform:~/os/第八次$ ./sleeping_barber
理发师1线程创建成功
理发师1线程准备进入临界区（等待顾客）
顾客1线程创建成功
顾客1线程准备进入临界区（进店）
顾客1线程已进入临界区（进店）
顾客1进店，当前等待顾客数：1，等待椅空闲数：2，等待理发
顾客1开始接受理发师1服务
顾客1理发完成，离店
顾客1线程已离开临界区（离店）
理发师1线程已进入临界区（等待顾客）
理发师1唤醒，当前等待顾客数：0，等待椅空闲数：3
理发师1开始为顾客理发，预计时长：1秒
顾客2线程创建成功
理发师1完成理发
理发师1线程已离开临界区（完成理发）
理发师1线程准备进入临界区（等待顾客）
顾客2线程准备进入临界区（进店）
顾客2线程已进入临界区（进店）
顾客2进店，当前等待顾客数：1，等待椅空闲数：2，等待理发
顾客2开始接受理发师1服务
```

代码块

- 1 理发师1线程创建成功
- 2 顾客1线程创建成功
- 3 顾客1线程准备进入临界区（进店）
- 4 顾客1线程已进入临界区（进店）
- 5 顾客1进店，当前等待顾客数：1，等待椅空闲数：2，进入等待队列
- 6 理发师1线程准备进入临界区（等待顾客）
- 7 理发师1线程已进入临界区（等待顾客）
- 8 理发师1被唤醒，当前等待顾客数：0，等待椅空闲数：3
- 9 理发师1开始为顾客理发，预计时长：4秒
- 10 顾客1开始接受理发师1服务
- 11 顾客2线程创建成功
- 12 顾客2线程准备进入临界区（进店）
- 13 顾客2线程已进入临界区（进店）
- 14 顾客2进店，当前等待顾客数：1，等待椅空闲数：2，进入等待队列
- 15 顾客2线程已离开临界区（离店）
- 16 顾客3线程创建成功
- 17 顾客3线程准备进入临界区（进店）
- 18 顾客3线程已进入临界区（进店）


```
19  顾客3进店，当前等待顾客数：2，等待椅空闲数：1，进入等待队列
20  顾客3线程已离开临界区（离店）
21  顾客4线程创建成功
22  顾客4线程准备进入临界区（进店）
23  顾客4线程已进入临界区（进店）
24  顾客4进店，当前等待顾客数：3，等待椅空闲数：0，进入等待队列
25  顾客4线程已离开临界区（离店）
26  顾客5线程创建成功
27  顾客5线程准备进入临界区（进店）
28  顾客5线程已进入临界区（进店）
29  顾客5进店，无空闲等待椅，立即离店
30  顾客5线程已离开临界区（离店）
31  ...
32  理发师1完成理发
33  理发师1线程已离开临界区（完成理发）
34  顾客1理发完成，离店
35  顾客1线程已离开临界区（离店）
36  理发师1线程准备进入临界区（等待顾客）
37  理发师1线程已进入临界区（等待顾客）
38  理发师1被唤醒，当前等待顾客数：2，等待椅空闲数：1
39  理发师1开始为顾客理发，预计时长：7秒
40  顾客2开始接受理发师1服务
41  ...
42  顾客30线程创建成功
43  顾客30线程准备进入临界区（进店）
44  顾客30线程已进入临界区（进店）
45  顾客30进店，当前等待顾客数：1，等待椅空闲数：2，进入等待队列
46  顾客30线程已离开临界区（离店）
47  理发师1完成理发
48  理发师1线程已离开临界区（完成理发）
49  顾客30理发完成，离店
50  顾客30线程已离开临界区（离店）
51  理发师1已服务所有30名顾客，下班！
52  所有线程执行完毕，程序正常退出
```

（二）结果分析

20. 同步机制有效性：

- 互斥量 `mutex` 确保 `waiting` 变量的原子操作：多个顾客线程同时进店时，仅一个线程能修改 `waiting`，避免了“同时判断等待椅空闲但实际超员”的竞态条件；
- 信号量 `customer` 与 `barber` 实现精准同步：无顾客时理发师阻塞在 `sem_wait(&customer)`，避免忙等待；顾客通过 `sem_post(&customer)` 唤醒理发师，通过 `sem_wait(&barber)` 等待服务，无信号丢失或误唤醒。

21. 场景约束满足情况：

- 等待椅数量严格控制： `waiting` 变量范围始终为0-3，当 `waiting=3` 时，后续顾客直接离店，符合“最多3人等待”的约束；
- 顾客总数控制：累计创建30个顾客线程，无重复或遗漏；所有顾客处理完毕后，理发师通过 `serve_count` 统计退出，边界处理正确；
- 随机特性实现：顾客到达间隔（1-3秒）和理发时长（1-10秒）通过随机函数生成，模拟真实场景。

22. 输出规范性：

- 严格按照要求输出四类日志：临界区进出日志、理发师状态日志（唤醒、理发开始/结束、下班）、顾客状态日志（进店、等待、理发、离店）、等待椅状态日志（等待数、空闲数）；
- 时序逻辑清晰：先进入临界区再修改状态，先理发再离店，无逻辑错乱。

五、问题与解决

遇到的问题	问题原因	解决方案
编译报错“undefined reference to pthread_create”	未链接Pthread线程库	编译命令添加 <code>-lpthread</code> 参数，即 <code>gcc sleeping_barber.c -o sleeping_barber -lpthread</code>
顾客编号重复	直接使用循环变量 <code>i</code> 作为顾客编号，线程共享变量	为每个顾客线程动态分配 <code>id</code> 内存（ <code>int *id = malloc(sizeof(int))</code> ），避免共享冲突
理发师未正常退出，陷入无限循环	未统计服务顾客数，无法判断是否完成所有服务	添加静态变量 <code>serve_count</code> ，统计服务次数，达到30次后退出循环
等待椅状态显示错误	未在临界区修改 <code>waiting</code> 变量，存在竞态条件	将 <code>waiting</code> 变量的读写操作包裹在 <code>pthread_mutex_lock/unlock</code> 之间
顾客唤醒理发师后未等待，直接离店	缺少 <code>sem_wait(&barber)</code> ，未同步理发师状态	顾客加入等待队列后，通过 <code>sem_wait(&barber)</code> 阻塞，直至理发师空闲

六、实验总结与扩展

（一）实验总结

23. 核心收获：

- 掌握了Posix信号量（`sem_init` / `sem_wait` / `sem_post` / `sem_destroy`）和Pthread互斥量（`pthread_mutex_*`）的完整使用流程，理解了同步与互斥的本质区别；
- 学会了多线程编程的关键操作：线程创建、参数传递（动态内存分配避免共享冲突）、线程等待（`pthread_join`）、资源销毁；
- 成功解决了经典同步问题，能够识别并规避竞态条件、忙等待等常见多线程问题。

24. 实验不足：

- 输出方式仅为文本日志，缺乏可视化界面，难以直观观察线程运行状态；
- 未处理线程创建失败、信号量初始化失败等异常场景的优雅退出；
- 仅支持1名理发师，扩展性有限。

（二）扩展思考

25. 多理发师场景扩展：

- 增加理发师数量（如2名）和理发椅数量（如2把），需定义信号量数组 `barber[N]`（N为理发师数），每个信号量对应一名理发师的空闲状态；
- 等待椅资源竞争仍通过互斥量保护，顾客唤醒任意空闲理发师，或按顺序等待。

26. 可视化输出优化：

- 使用Linux `ncurses` 库实现图形化界面，实时显示理发椅状态（空闲/忙碌）、等待椅占用情况、理发师/顾客位置；
- 用不同颜色标注线程状态（等待/运行/结束），提升可读性。

27. 异常处理增强：

- 为信号量初始化、线程创建等操作添加错误处理，通过 `perror` 输出错误信息，调用 `exit` 或 `pthread_cancel` 优雅释放资源；
- 处理顾客线程中途退出的情况，避免 `waiting` 变量计数错误。

七、实验结论

本实验基于Posix信号量和Pthread互斥量，成功实现了睡眠理发师问题的多线程解决方案。实验结果表明：

28. 同步机制设计合理，无竞态条件、无忙等待，满足线程安全要求；

29. 场景约束（1名理发师、3把等待椅、30名顾客）全部满足，边界处理正确；

30. 输出日志完整规范，时序逻辑清晰，符合实验要求。

通过本次实验，深入理解了多线程同步与互斥的核心原理，掌握了多线程编程的实践技巧，为后续复杂并发程序开发奠定了基础。

附录：完整代码文件

代码块

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <time.h>
6  #include <unistd.h>
7
8  #define CHAIRS 3          // 等待椅数量
9  #define TOTAL_CUSTOMERS 30 // 总顾客数
10 sem_t customer;          // 顾客到达信号量（唤醒理发师）
11 sem_t barber;            // 理发师可用信号量
12 pthread_mutex_t mutex;    // 保护等待顾客数的互斥量
13 int waiting = 0;          // 当前等待顾客数
14 int barber_id = 1;        // 理发师编号
15
16 // 理发师线程函数
17 void *barber_thread(void *arg) {
18     while (1) {
19         printf("理发师%d线程准备进入临界区（等待顾客）\n", barber_id);
20         sem_wait(&customer);
21         printf("理发师%d线程已进入临界区（等待顾客）\n", barber_id);
22
23         pthread_mutex_lock(&mutex);
24         waiting--;
25         printf("理发师%d被唤醒，当前等待顾客数： %d，等待椅空闲数： %d\n",
26             barber_id, waiting, CHAIRS - waiting);
27         pthread_mutex_unlock(&mutex);
28
29         int cut_time = rand() % 10 + 1;
30         printf("理发师%d开始为顾客理发，预计时长： %d秒\n", barber_id, cut_time);
31         sleep(cut_time);
32         printf("理发师%d完成理发\n", barber_id);
33
34         sem_post(&barber);
35         printf("理发师%d线程已离开临界区（完成理发）\n", barber_id);
36
37         static int served_count = 0;
38         served_count++;
```

```

39         if (served_count == TOTAL_CUSTOMERS) {
40             printf("理发师%d已服务所有%d名顾客，下班! \n", barber_id,
TOTAL_CUSTOMERS);
41             break;
42         }
43     }
44     pthread_exit(NULL);
45 }
46
47 // 顾客线程函数
48 void *customer_thread(void *arg) {
49     int id = *(int *)arg;
50     free(arg);
51
52     printf("顾客%d线程准备进入临界区（进店） \n", id);
53     pthread_mutex_lock(&mutex);
54     printf("顾客%d线程已进入临界区（进店） \n", id);
55
56     if (waiting < CHAIRS) {
57         waiting++;
58         printf("顾客%d进店，当前等待顾客数：%d，等待椅空闲数：%d，进入等待队列\n",
59             id, waiting, CHAIRS - waiting);
60         pthread_mutex_unlock(&mutex);
61
62         sem_post(&customer);
63         sem_wait(&barber);
64
65         printf("顾客%d开始接受理发师%d服务\n", id, barber_id);
66     } else {
67         printf("顾客%d进店，无空闲等待椅，立即离店\n", id);
68         pthread_mutex_unlock(&mutex);
69         printf("顾客%d线程已离开临界区（离店） \n", id);
70         pthread_exit(NULL);
71     }
72
73     printf("顾客%d理发完成，离店\n", id);
74     printf("顾客%d线程已离开临界区（离店） \n", id);
75     pthread_exit(NULL);
76 }
77
78 int main() {
79     pthread_t barber_tid;
80     pthread_t customer_tids[TOTAL_CUSTOMERS];
81     int i;
82
83     if (sem_init(&customer, 0, 0) == -1 || sem_init(&barber, 0, 1) == -1) {
84         perror("sem_init failed");

```

```

85     exit(EXIT_FAILURE);
86 }
87 if (pthread_mutex_init(&mutex, NULL) == -1) {
88     perror("pthread_mutex_init failed");
89     exit(EXIT_FAILURE);
90 }
91
92 srand((unsigned int)time(NULL));
93
94 if (pthread_create(&barber_tid, NULL, barber_thread, NULL) == -1) {
95     perror("pthread_create barber failed");
96     exit(EXIT_FAILURE);
97 }
98 printf("理发师%d线程创建成功\n", barber_id);
99
100 for (i = 0; i < TOTAL_CUSTOMERS; i++) {
101     int *id = malloc(sizeof(int));
102     *id = i + 1;
103     if (pthread_create(&customer_tids[i], NULL, customer_thread, id) ==
-1) {
104         perror("pthread_create customer failed");
105         exit(EXIT_FAILURE);
106     }
107     printf("顾客%d线程创建成功\n", *id);
108     sleep(rand() % 3 + 1);
109 }
110
111 for (i = 0; i < TOTAL_CUSTOMERS; i++) {
112     if (pthread_join(customer_tids[i], NULL) == -1) {
113         perror("pthread_join customer failed");
114         exit(EXIT_FAILURE);
115     }
116 }
117
118 if (pthread_join(barber_tid, NULL) == -1) {
119     perror("pthread_join barber failed");
120     exit(EXIT_FAILURE);
121 }
122
123 sem_destroy(&customer);
124 sem_destroy(&barber);
125 pthread_mutex_destroy(&mutex);
126
127 printf("所有线程执行完毕，程序正常退出\n");
128 return 0;
129 }

```

